Challenges I Faced Building the States REST API

INF653 Final Project, Node.js / Express / MongoDB

This project was smoother than the midterm in some places and rougher in others. Here are the spots that actually gave me trouble.

Two data sources, one response

The trickiest part for me was that the data lives in two places. statesData.json has the demographic info, but funfacts have to come from MongoDB. My first instinct was to dump everything into Mongo so I only had to query one place, but the requirements explicitly say not to do that. So I read the JSON first, then pull matching records from Mongo and attach the funfacts to a copy of the JSON object. The copy part matters. My first version started returning funfacts on states that shouldn't have had any, because require()'d JSON gets cached and mutating it once changes it for every future request. Switching to a shallow copy with the spread operator fixed it.

Mongoose silently ignoring array updates

PATCH and DELETE both edit a single element inside the funfacts array. I assumed doc.funfacts[i] = newValue followed by save() would work, but Mongoose just refused to save the change. The document looked right in memory, save() returned without errors, but the database still had the old value. After some googling I found out Mongoose doesn't always notice in-place mutations on array paths. Calling doc.markModified('funfacts') before save() fixed it. For DELETE I used filter() instead of splice or delete on purpose, so I never leave an undefined slot behind that the random funfact endpoint could grab later.

Order of validation matters more than I expected

The PATCH endpoint has four ways it can fail: missing index, missing funfact value, the state has no facts at all, or the index is out of range. Each one needs its own specific error message. The order I check things matters, because a request with no index would get the wrong error if I checked "state has no facts" first, and that would fail the autograder even though the request was technically rejected. So the order is: check index exists, check funfact exists, check the state has any facts, then check the index is in range.

404 with both HTML and JSON

The catch-all 404 has to return HTML or JSON depending on the Accept header. Express's req.accepts() helper makes this pretty easy. The gotcha is that you have to call res.status(404) *before* res.sendFile() or res.json(). Set the status after and it gets overwritten back to 200, because sendFile and json end the response.

Glitch is gone

The course materials say to deploy to Glitch, but Glitch shut down project hosting in July 2025. I used Render instead, which is what the professor's example API runs on anyway. The deployment was straightforward, but the free tier puts services to sleep after 15 minutes of no traffic and cold starts take about 30 seconds, which would tank the autograder if I didn't warm it up first. The .env file with my Mongo connection string is gitignored, and Render has its own environment variable panel where I pasted it after deploying.